

A Fast Quasi-Linear Heuristic for the Close-Enough Traveling Salesman Problem

1 Introduction

The *Close-Enough Traveling Salesman Problem* (CETSP) is a continuous generalization of the classical Traveling Salesman Problem (TSP), where each target is associated with a circular neighborhood that the salesman must intersect rather than a precise point. The CETSP thus combines the discrete optimization of the visiting sequence with the continuous optimization of visiting positions, and it naturally models path-planning tasks such as radio meter reading, drone-based inspection, and robotic welding (LeiHao2024; DiPlacido2022).

Despite recent progress, most high-performing CETSP algorithms rely on population-based or local-improvement metaheuristics that achieve strong solution quality at the expense of high computational cost. In this work, we present a new heuristic for the CETSP that is inspired by the quasi-linear *pair-center algorithm* for the Euclidean TSP proposed by Formella2024. While originally designed for point-based TSP instances, the pair-center concept extends naturally to the CETSP once disk geometry and intersection logic are handled properly. Our resulting method runs in expected $O(n \log n)$ time, making it markedly faster and more scalable than current state-of-the-art CETSP solvers. Although it does not seek to outperform metaheuristic approaches in final tour length, its runtime efficiency allows application to extremely large or real-time instances that remain intractable for existing techniques.

2 Related Work

3 Algorithm

Our approach follows the overall structure of the quasi-linear *pair-center algorithm* introduced by Formella2024 for the Euclidean TSP, but extends it to handle circular neighborhoods as in the Close-Enough Traveling Salesman Problem. Conceptually, the method retains the same two-phase organization: a *clustering phase* that builds a hierarchical representation of the instance, and a *construction phase* that incrementally expands a feasible tour from this hierarchy.

In the clustering phase, the algorithm recursively merges the closest pair of geometric objects into a new composite “proxy” object until a single hierarchy remains. In the CETSP setting, these objects are circles rather than points as in Formella2024. The result is a binary clustering tree whose internal nodes represent proxy circles.

The construction phase then traverses this tree to build a closed tour. As in the original pair-center approach, the tour is dynamically maintained so that it remains feasible at every step. However, the continuous nature of the CETSP introduces two additional challenges: first, multiple circles may correspond to a single effective tour point when their feasible regions overlap; second, each tour point admits continuous local optimization within its circle. To address these, the algorithm performs lightweight dynamic updates—reinserting or locally re-optimizing tour points—while preserving near-linear expected runtime.

Overall, the method retains the speed and structural simplicity of the pair-center algorithm while incorporating the geometric flexibility required for close-enough constraints. Detailed definitions, data structures, and optimization steps are presented in the following sections.

3.1 Data Structures

We make extensive use of geometric search structures to maintain and query spatial relationships efficiently. In particular, we employ an R*-tree to store bounding boxes of the current set of circles, enabling logarithmic-time expected queries for nearest neighbors, insertions, and deletions. The resulting operations have expected $O(n \log n)$ runtime; see **Formella2024** for a detailed review and analysis.

3.2 Clustering Phase

The clustering phase closely follows the structure of the pair-center algorithm described by **Formella2024**, with appropriate modifications to handle circular regions instead of point targets. As in the original approach, the goal is to construct a hierarchical binary tree over all targets by iteratively merging the closest pair of geometric objects, replacing them with a single representative (a *proxy circle*). The leaves of the resulting tree correspond to the original input disks, and each internal node stores its proxy circle and the associated merge distance.

Preprocessing. Before clustering, we remove redundant circles—specifically, any circle that is completely contained within another. This is performed by approximating each circle with its axis-aligned bounding box, sorting all boxes by decreasing radius, and using the R*-tree to query for boxes that fully cover the current one. To mitigate alignment bias from axis-aligned boxes, we randomize the instance by applying a random global rotation to all circle centers. This randomization slightly improves expected merging quality while maintaining near-linear preprocessing cost.

Merge Step. Let c_1 and c_2 denote the two circles chosen for merging. Their *effective distance* is defined as

$$d(c_1, c_2) = \|p_1 - p_2\| - r_1 - r_2,$$

that is, the Euclidean gap between their boundaries. Intuitively, smaller values indicate strong overlap or proximity, which makes the pair more representative for early merging.

The merge procedure operates as follows:

1. Select the pair (c_1, c_2) with the minimal $d(c_1, c_2)$ value.
2. Remove c_1 and c_2 from the R*-tree and from the corresponding neighbor structures.
3. Compute their proxy circle c' .
4. Insert c' into the R*-tree and update all relevant neighbor lists.
5. Insert c' as an internal node in the binary tree, with c_1 and c_2 as its children.

To maintain efficiency, we adopt the same general mechanism as **Formella2024**. Each active circle keeps a list of its current nearest neighbors, and a global min-heap stores tuples of the form $(d(c_i, c_j), i, j)$. When a pair is merged, only the affected neighbors of c_i and c_j are updated.

Approximate Neighbor Search. Because the effective distance $d(c_1, c_2) = \|p_1 - p_2\| - r_1 - r_2$ depends on both position and radius, exact nearest-neighbor search under this metric is impractical. Instead, we perform approximate search by querying the k (small constant) closest bounding boxes (from a certain center p_1) and evaluating $d(c_1, c_2)$ explicitly for those candidates. This is inexact in two ways: boxes do not perfectly encapsulate the circles, and the distance any value of $\|p_1 - p_2\| - r_2$ not greater than 0 are treated the same. However, in practice, small constant values of k suffice to preserve both runtime efficiency and clustering quality.

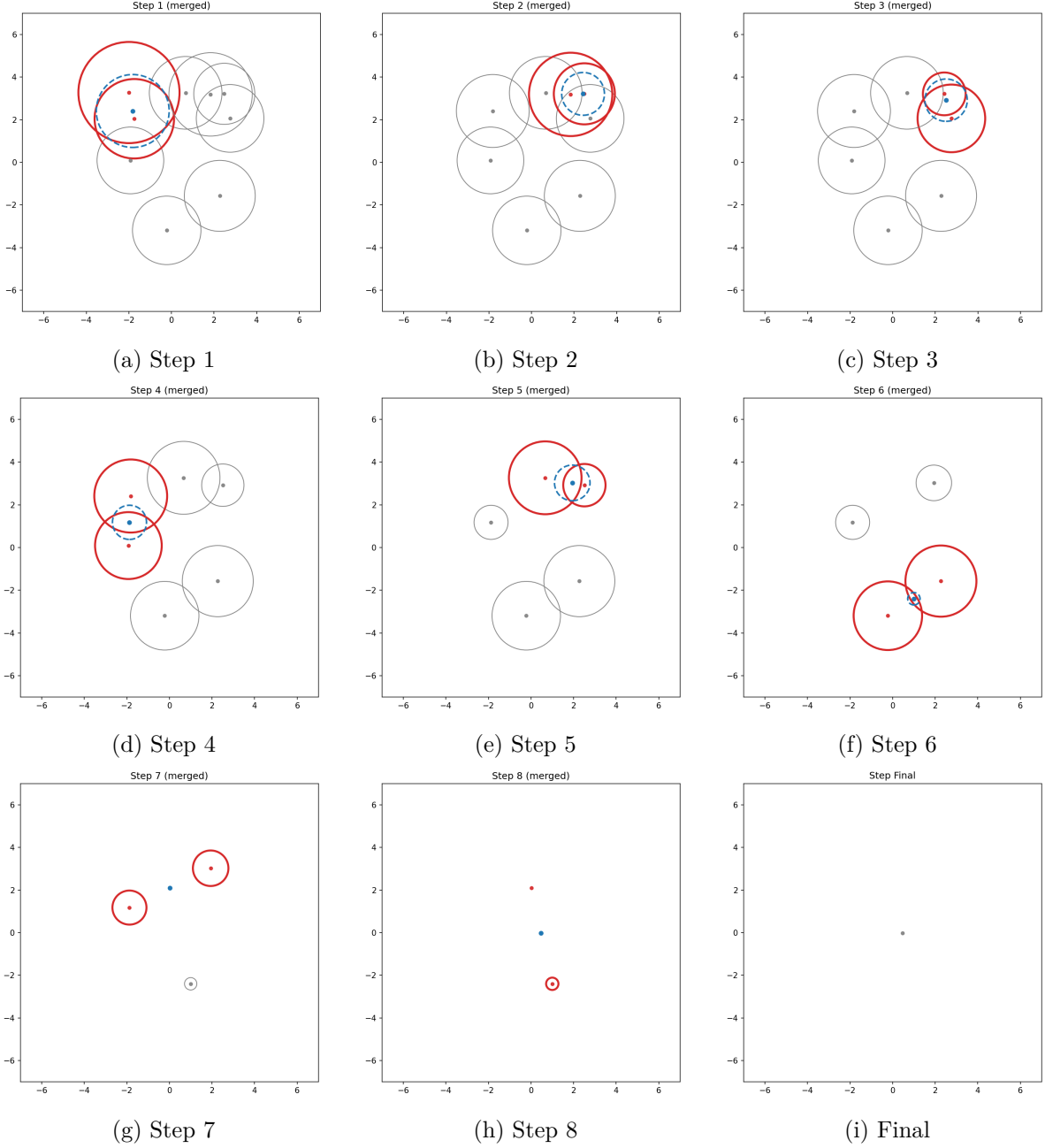


Figure 1: Visualization of the clustering phase for a sample instance with $n = 9$ disks. Each subfigure shows one merge step, where the two closest circles (highlighted) are replaced by their proxy circle (dashed blue outline). The final frame (bottom right) shows the single remaining cluster.

Proxy Circle Computation. When merging two circles $c_1 = (p_1, r_1)$ and $c_2 = (p_2, r_2)$, we define their *proxy circle* $c' = (p', r')$ as a compact approximation of the lens-shaped region covered by their union. In contrast to simply using the midpoint between centers, our implementation constructs c' geometrically along the line connecting p_1 and p_2 .

Specifically, let $d = \|p_1 - p_2\|$ denote the center distance. If a circle covers another circle, we simply take the smaller circle. If the circles do not intersect at all ($d \geq r_1 + r_2$), we take the midpoint between the circle boundaries and radius 0 (i.e. a single point).

Otherwise, we construct a proxy circle. We determine the two points where the line connecting p_1 and p_2 intersects the boundaries of the two circles, and place the new center p' at the midpoint

between these boundary points. The resulting circle approximates the geometric “lens” formed by the intersection. We compute both the overlap depth

$$\delta = \frac{1}{2}(r_1 + r_2 - d)$$

and the half-chord length

$$h = \sqrt{r_1^2 - a^2}, \quad a = \frac{r_1^2 - r_2^2 + d^2}{2d},$$

which jointly define a plausible range for the new radius. To introduce mild stochastic diversity, the final radius r' is drawn uniformly from the interval $[\delta, h]$. This randomization helps avoid degenerate clustering patterns and allows multiple independent runs to explore slightly different hierarchical structures.

Although this proxy construction is heuristic, it consistently yields stable and geometrically balanced merges in practice, while maintaining constant-time computation and preserving the algorithm’s overall quasi-linear runtime.

3.3 Construction Phase

The construction phase of our algorithm follows the overall strategy of the pair-center algorithm described by **Formella2024**, with modifications to handle circular regions instead of point targets. The core objective is to insert each original circle into the tour by iteratively decomposing the hierarchical structure, maintaining a closed tour where a single tour point can represent multiple covered circles.

Tour Initialization and Expansion. The process begins by initializing the tour with a single tour point corresponding to the center point of the root of the binary tree. The root proxy circle itself is inserted into a global max-heap, which is ordered by the merge distance stored at the center node. The algorithm proceeds iteratively, expanding the subtrees until the max-heap is empty.

Circle Decomposition and Removal. In each step, the maximal entry in the heap—a circle $c = (O, r)$ —is popped from the heap. This circle c is then removed from the tour structure. This removal is handled conditionally: if the tour point that contained c represented only this single circle, the tour point is entirely removed from the sequence. If the tour point represented a list of multiple circles (including c), only c is removed from that list, and the tour point remains active.

Insertion of Generating Circles. The process then focuses on inserting the two generating circles, c_i and c_j , that formed the proxy circle c . Insertion is performed by checking two possibilities:

- **Check existing points:** The first priority is to assign the circle to an existing tour point to incur zero additional travel cost. This is possible if the existing tour point is not greater than the circle’s radius. An R-tree of tour points is used for efficient querying of suitable existing points.
- **Create new tour point:** If no existing point can cover the circle, a new tour point P must be created inside the circle (O, r) and inserted into an existing tour segment AB . The optimal placement of P must minimize the added tour length, $\Delta L = |AC| + |BC| - |AB|$.

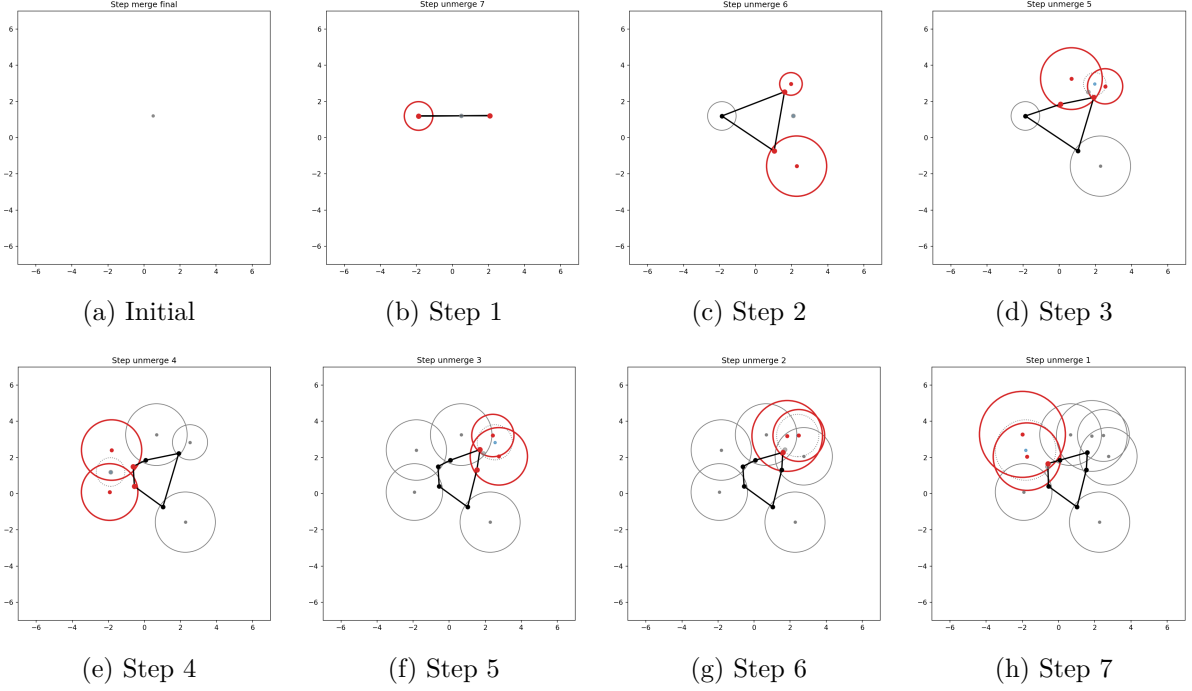


Figure 2: Visualization of the construction phase for the same sample instance. The process starts from the final proxy circle obtained in the clustering phase (top left) and iteratively expands proxy circles into their generating circles while updating the tour. Newly inserted circles and tour points are highlighted, while removed proxy elements are shown in a faded, dashed style. Note that we do not show point optimizations here.

Approximate Tour Placement. Finding the globally optimal insertion point is computationally intensive (this is Alhazen’s problem). Therefore, we use an approximation: we first select the k nearest tour segments AB using an R-tree of tour segments. For each segment, we put a provisional new point P at the bisection of the angle $\angle AOB$, where O is the circle center. We then compare all points P to find the best-fitting segment. This method is a fast and effective approximation of the optimal point (LeiHao2024). Optionally, we can also apply a single Newton-Raphson iteration to refine this approximate placement. If the generating circles c_i or c_j are themselves proxy circles (internal nodes), they are inserted into the max-heap for further hierarchical expansion. The algorithm terminates once the max-heap is empty, indicating that all original leaf-node circles have been assigned to tour points, resulting in a complete tour.

The reoptimization problem is as follows. Given a tour segment (A, B) and a tour point P representing a set of circles $\{(O_i, r_i)\}$, we wish to find a new position P^* within the feasible intersection region of these circles that minimizes

$$\Delta L = |AP^*| + |BP^*| - |AB|.$$

This optimization is nontrivial even for a single circle, and becomes analytically intractable for multiple overlapping disks. We therefore adopt a fast geometric approximation that suffices for local improvement.

1. If the segment AB intersects the common feasible region of all circles, the optimal solution incurs zero additional cost; in this case, we simply take a point in the intersection.
2. Otherwise, the optimal P^* must lie on the boundary of the intersection region. We approximate this by performing a single gradient-descent step: we compute the gradient of ΔL with respect to P and move P maximally along that direction until it would leave any of the circles. This yields a fast, stable adjustment that effectively reduces local tour length.

To control runtime, reoptimization is triggered according to a simple exponential schedule. Specifically, a tour point is reoptimized whenever the number of insertions of circles corresponding to that point reaches 2^n for some integer n . This ensures that frequently updated regions receive more attention while overall computational effort remains bounded.

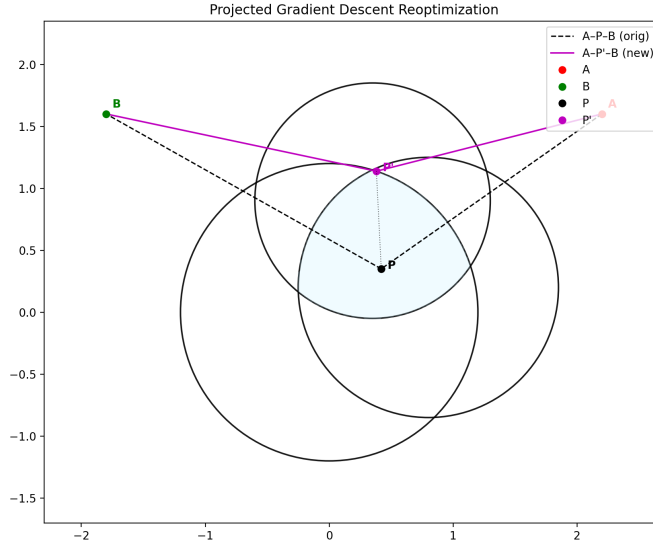


Figure 4: Illustration of the point reoptimization step. The new position P^* is obtained by projecting the current point P toward the gradient direction of decreasing tour cost, constrained to remain within all corresponding circles.

Together, these two mechanisms maintain geometric consistency of the tour and correct local inefficiencies introduced during hierarchical construction. Both procedures are incremental, spatially localized, and amortized over the construction process, maintaining the total runtime while providing measurable quality improvements.

3.5 Runtime Analysis

We analyze the expected running time of the algorithm. As in the clustering phase, we measure time in terms of basic geometric operations (R-tree queries, insertions, deletions, and heap

operations). We assume the standard randomized/average-case performance bounds for R-trees (or similar balanced spatial search structures): each query, insertion or deletion takes expected $O(\log n)$ time. All claims below are in expectation with respect to the randomization used in preprocessing and the probabilistic behavior of the spatial index.

Single-query cost. Retrieving the k nearest bounding boxes (or segments or points) from the R-tree costs expected $O(\log n + k)$. We repeatedly use this fact to bound the cost of approximate nearest-neighbor and segment queries.

3.5.1 Preprocessing phase

We first sort the n axis-aligned bounding boxes by decreasing radius (or equivalently by the contained circle radius). This sorting costs $O(n \log n)$.

After sorting, we process each box once: for the current box we query the R-tree for the k nearest boxes (constant k returned by the spatial index) and test those candidates for full containment; any box found to be fully contained is removed. Let k_i denote the number of candidates examined for the i -th query. The total query cost is

$$\sum_{i=1}^n (O(\log n) + k_i) = O(n \log n) + \sum_{i=1}^n k_i.$$

It remains to bound $\sum_i k_i$ in expectation. By the random global rotation applied during preprocessing, an axis-aligned covering box has a constant (geometric) probability that its inscribed circle also covers the target circle; a simple planar-angle argument gives this probability as a positive constant $= \pi/4$. Hence each successful removal corresponds in expectation to only a constant number of candidate examinations. Since at most n boxes can be removed overall, we obtain $E[\sum_i k_i] = O(n)$.

Combining terms yields an expected preprocessing cost of

$$T_{\text{preproc}} = O(n \log n) + O(n) = O(n \log n).$$

3.5.2 Clustering phase

The clustering phase performs $n - 1$ merge iterations. In each iteration we:

- extract the current closest pair from a min-heap (expected $O(\log n)$),
- remove two entries from the R-tree (expected $O(\log n)$ each),
- compute a proxy circle (constant-time geometric work),
- insert the proxy into the R-tree (expected $O(\log n)$), and
- update a constant number of neighbor lists (each update involves at most a constant number of R-tree / heap operations and therefore costs expected $O(\log n)$).

Summing the dominant $O(\log n)$ costs over the $O(n)$ merges yields

$$T_{\text{cluster}} = O(n \log n).$$

3.5.3 Construction phase (excluding point-optimizations)

During construction we perform $O(n)$ heap pops and insertions (each costing expected $O(\log n)$), and for each insertion we perform a bounded number of R-tree queries or updates (nearest-point checks, segment queries, and structure updates), each costing expected $O(\log n)$. Thus the dominant cost of construction (excluding point-optimizations) is

$$T_{\text{construct}} = O(n \log n).$$

3.5.4 Cost of reinserts

During the construction phase, we also use the point reinsertion mechanism described above. Each circle reinserted costs $O(\log n)$ expected time, as described in the construction phase. To bound the runtime of our reinsertion policy, we must bound the total number of reinsertions performed. Note that this bound depends on the number of *circles* that are reinserted, not the number of *tour points* that are reinserted.

We prove this using a *gadget problem*. In this gadget problem, there are no circle deletions and energy subtractions can go into any node, not just neighbors. This also allows us to discount the tour structure entirely, and allow our process to work on an infinite set of abstract nodes. Note that these changes can only increase the number of reinsertions performed, so a bound on the gadget problem also bounds our actual reinsertion cost.

Gadget problem description. We consider a collection of nodes (finite or infinite in number), each of which is associated with two integer-valued parameters:

$$E(v) \in \mathbb{Z}_{\geq 0} \quad (\text{energy}), \quad W(v) \in \mathbb{Z}_{\geq 0} \quad (\text{weight}).$$

Initially, all nodes satisfy $E(v) = W(v) = 0$.

We define an *operation* as the following sequence of updates:

1. *Insertion.* Select a node v_+ and perform the update

$$E(v_+) += 3, \quad W(v_+) += 1.$$

2. *Energy reduction.* Then, twice, select (possibly different) nodes v_- and perform the update

$$E(v_-) -= 1.$$

Whenever a node v reaches energy $E(v) = 0$, we say that v *resets*. Let $w = W(v)$ be its weight at the moment of reset. Upon reset, we perform:

$$W(v) \leftarrow 0,$$

and immediately execute w new operations as defined above. These induced operations are called *extra operations*.

More concretely, imagine a stack of updates. Initially, we push $3n$ updates onto the stack, alternating between two energy reductions and one insertion. We then repeatedly pop updates from the stack and execute them. Whenever an energy reduction causes a node to reset, we push $3w$ new operations onto the stack in the same manner, where w is the weight of the node at reset time. The total number of operations is equal to the number of insertions performed.

Suppose we begin with n *original operations* (executed according to the above rule, including all recursively spawned extra operations). Let X denote the total number of extra operations generated in the entire process. Show that the process always terminates (i.e. X is finite) and prove that $X \in O(n)$.

Proof of gadget problem bound. Start with n initial operations. Let X be the total number of extra operations ever spawned. Then the process terminates and

$$X \leq 2n.$$

We proceed by induction on the number of initial operations performed.

Base case: $k = 1$. Execute the first initial operation: it does one $(+3)$ (on some node) and two (-1) updates. No node can reach energy 0 as a result of this single operation, because the

only possible decrements are the two (-1) s produced by this operation (they can reduce the node that received $(+3)$ by at most 2, leaving energy ≥ 1). Hence no extra operation can be spawned. So after the first initial op finishes (including any extras it might spawn—there are none) we have a finite prefix, and the number of extra ops produced so far $X_1 = 0 \leq 2 \cdot 1$.

Inductive hypothesis. Fix $k \geq 1$. Assume that after performing the first k initial operations (together with all extra ops they spawn) the process finishes a finite prefix and the total number of extras so far satisfies

$$X_k \leq 2k.$$

Inductive step. Suppose for contradiction that after performing the first $k + 1$ initial operations (and following all cascading extra ops) either (a) the process does not terminate, or (b) it terminates but the total number of extra ops X_{k+1} produced by the first $k + 1$ initial ops satisfies $X_{k+1} > 2(k + 1)$.

If (a) holds, then the number of extra ops produced is infinite; in particular there is a finite time at which the count of extra ops exceeds $2(k + 1)$. Thus in both (a) and (b) we can find a finite prefix of the run that contains the first $k + 1$ initial operations and whose number of extra ops X' satisfies

$$X' > 2(k + 1).$$

Choose the earliest (i.e., shortest) such finite prefix—earliest in the sense of the first time during the run the extra-op count becomes $> 2(k + 1)$. By construction this prefix is finite and contains exactly $(k + 1) + X'$ executed operations (initial + extra).

Now count updates in that prefix:

- Every executed operation contributes one $(+3)$ and two (-1) updates. So the prefix contains total $(+3)$ -events equal to $(k + 1) + X'$ and total (-1) -events equal to $2((k + 1) + X')$.
- Each extra operation in the prefix corresponds to a reset that consumed a weight unit which in turn came from a prior $(+3)$. To produce one extra operation (i.e. to consume one weight-unit) the corresponding $(+3)$ must have been fully drained by three (-1) decrements on that node. Thus the decrements that were used to create these X' extra ops are at least $3X'$ in number.

But these decrements must be among the $2((k + 1) + X')$ total decrements that actually occurred in the prefix. Therefore

$$3X' \leq 2((k + 1) + X').$$

Rearranging:

$$3X' \leq 2k + 2 + 2X' \implies X' \leq 2(k + 1).$$

This contradicts $X' > 2(k + 1)$, so our assumption was false.

Therefore neither (a) nor (b) can happen: the run started by the first $k + 1$ initial operations (together with all extras they spawn) must terminate and its number of extra ops satisfies $X_{k+1} \leq 2(k + 1)$. This completes the inductive step.

By induction, for every $k \leq n$ the prefix that contains the first k initial operations (and all extras they spawn) is finite and the number of extra operations produced satisfies $X_k \leq 2k$. Applying this to $k = n$ gives termination for the whole process and the desired bound

$$X \leq 2n,$$

so $X = O(n)$.

3.5.5 Cost of point reoptimizations

A reoptimization for a tour point operates only on the (usually small) list of circles associated to that point. By our exponential scheduling (reoptimize after $1, 2, 4, \dots$ insertions), the total amount of work spent on reoptimizing a single point over the entire run is $O(s)$ where s is the total number of insertions into that point; summing over all points yields an overall reoptimization cost that is proportional to the total number of insertions, i.e. $O(I) = O(n)$. Thus

$$T_{\text{reopt}} = O(n).$$

3.5.6 Total expected time

Combining the dominant terms,

$$T_{\text{total}} = O(n \log n).$$

Remarks and caveats. The $O(n \log n)$ bound hides constant factors coming from the k -nearest queries, box approximations, random rotations, and neighbor-list management; these are implementation-dependent but do not change the asymptotic claim.

4 Benchmark Evaluation and Comparison

4.1 Benchmark Instances

We evaluate our algorithm on the **Mennell2009** dataset; see **LeiHao2024** for an overview of its composition and benchmark usage in the CETSP literature.

These instances were originally obtained from three sources—**TSPLIB** (**TSPLIB1991**), **Teams**, and **Geometric**—and consist of a total of 62 benchmark cases:

- *TSPLIB subset.* 28 instances generated from seven graphs (**d493**, **dsj1000**, **kroD100**, **lin318**, **pcb442**, **rat195**, and **rd400**) with sizes ranging from 100 to 1000 nodes. These include two with uniformly generated nodes (**d493**, **dsj1000**), two with clustered nodes (**kroD100**, **lin318**), two drill-press problems (**pcb442**, **rat195**), and one with nodes arranged roughly along lines (**rd400**).
- *Teams subset.* 14 instances derived from seven graphs containing the terms *team* or *bonus*, introduced by **Gulczynski2006**, comprising three uniformly distributed and four clustered point sets.
- *Geometric subset.* 20 instances from **Mennell2009**, including 9 *Bubbles* (overlapping concentric squares), 5 *ConcentricCircles*, 5 *RotatingDiamonds* (nonoverlapping concentric squares), and one *chaoSingleDep* instance based on **Chao1993**.

Following **LeiHao2024**, the dataset can be grouped by overlap characteristics and disk radii:

- *Group 1 (G1).* 27 instances with different overlap ratios (OR), identical radii within each instance.
- *Group 2 (G2).* 21 instances with fixed overlap ratios (0.02, 0.1, and 0.3), again identical radii within each instance.
- *Group 3 (G3).* 14 instances with heterogeneous radii.

4.2 Dataset Discrepancies and Reconstruction

The *Mennell dataset* currently distributed on public repositories differs substantially from the version used in prior studies. In the current release,

- the *TSPLIB subset* no longer includes the identical-radius variants,
- several *new instances* (the `nlpProb` series) appear, and
- a *large-scale* 13509-circle test case (`usa13509.txt`) has been added.

This mismatch complicates reproducibility and benchmarking, as overlap ratios and radii are inconsistent across sources. Furthermore, the calculation of the *overlap ratio* (OR) is ambiguous in the literature: **Mennell2009** and **DiPlacido2022** describe different procedures for computing OR, and the provided data does not align with either definition.

To maintain compatibility with prior CETSP works, we reconstructed the original identical-radius test cases. Since the circle centers and solution tours are known, and the radii of the circles are equal, the radius can be recovered directly as

$$r = \max_i \min_j \|p_i - c_j\|, \quad (1)$$

where p_i are points on the optimal tour and c_j are the circle centers. Using this method, we successfully recovered all identical-radius test instances corresponding to those used by **LeiHao2024** and earlier studies.

4.3 Comparison Protocol

All experiments were conducted on the reconstructed benchmark suite, ensuring consistency with previous publications. We report results for each of the 62 canonical instances and compare our algorithm’s performance against the state-of-the-art CETSP solver **LeiHao2024**, adjusted for hardware differences. See **LeiHao2024** for how their solver compares to earlier methods.

For completeness, we also evaluate our method on the newer `nlpProb` and `usa13509` instances; however, since no prior results exist for these cases, it would be meaningless to include them in the main comparative analysis.¹

4.4 Result and Performance Analysis

Results are summarized in Table 1, which lists for each instance the best-known solution from **LeiHao2024**, our solver’s solution, and the percentage gap defined as

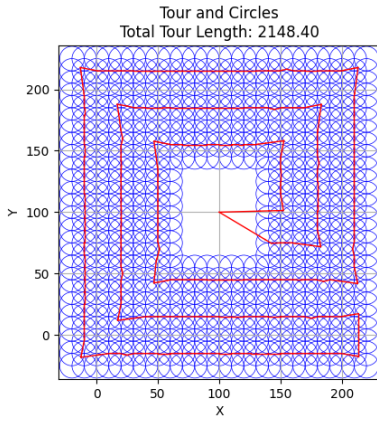
$$\text{Gap} = 100 \cdot \frac{\text{OurSolution} - \text{BestKnown}}{\text{BestKnown}}.$$

Overall, our method achieves solutions within **0–2%** of the best-known results for nearly all instances. An improvement is observed on one case (`bonus1000`, -1.49%), while the majority of gaps remain below 2%. This confirms that our algorithm performs competitively on a broad range of CETSP topologies.

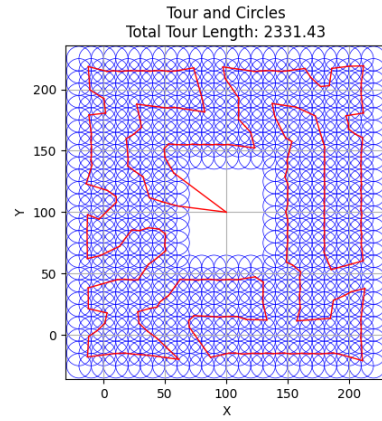
Structured-instance behavior. Two families of instances reveal the main limitations of our current approach:

¹The results are still available in the dataset for future benchmarking.

- **RotatingDiamonds series.** These instances exhibit geometric regularity and low overlap, making performance highly dependent on the quality of the underlying TSP solver. Our implementation inherits the $O(n \log n)$ heuristic of **Formella2024**, which yields near-linear scalability but cannot guarantee globally optimal TSP tours. Consequently, we observe gaps of up to 2.9% (e.g., `rotatingDiamonds5`). Further improvement would require integrating a better TSP solver.
- **Bubbles series.** These instances feature overlapping concentric structures where the optimal CETSP tours form quasi-spiral linear traversals. Our solver’s local search mechanisms are insufficient to escape shallow minima in such structured landscapes, and random initialization produces worse ‘jagged’ tours. Performance degradation increases with instance size, reaching 8.5% on `bubbles9`. Improvement here is hard and would likely require more sophisticated global optimization strategies.



(a) Benchmark solution (**LeiHao2024**)



(b) Our solver’s solution

Figure 5: Comparison of the benchmark and our solver’s tour for the `bubbles9` instance. The benchmark exhibits a highly structured, spiral traversal through the concentric layers of circles, whereas our solver’s path shows local inefficiencies and incomplete alignment along the optimal spiral pattern. This case highlights the solver’s difficulty escaping local minima in structured overlapping geometries.

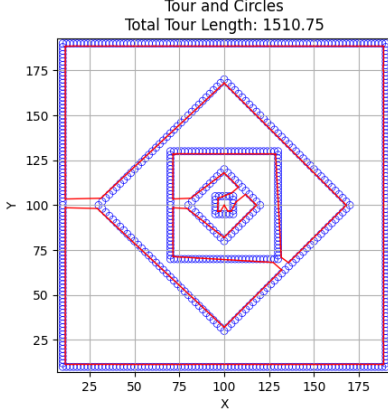
General performance. For all other categories—including *Teams*, *TSPLIB*, and *Concentric-Circles* instances—the gap remain consistently below 1%, matching or closely approaching the best-known solutions in the literature. The reconstructed identical-radius tests (*Group 1* and *Group 2*) show particularly stable convergence.

5 Runtime Experiments

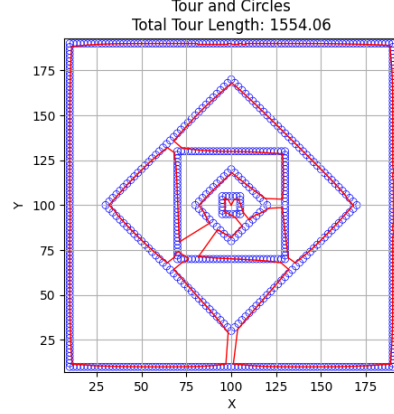
We conducted a series of empirical runtime measurements to study the scaling behavior of our CETSP solver.

5.1 Random Input Experiments

First, we generated random test instances as follows: for each of $n = 2^7, 2^8, \dots, 2^{20}$ circles, we chose the circle centers uniformly at random from the square $[-L, L]^2$, where L is a problem-dependent limit, and the radii uniformly from $[0.01, 0.02] \cdot L$. Each instance was solved with 10 repetitions to average out noise.



(a) Benchmark solution (**LeiHao2024**)



(b) Our solver's solution

Figure 6: Comparison of the benchmark and our solver's tour for the `rotatingDiamonds5` instance. The benchmark achieves a better traversal order across the concentric diamond layers, whereas our solver generates a worse traversal primarily due to suboptimal TSP ordering inherited from the base heuristic of **Formella2024**. Additional inefficiency arises from locally optimized point selections rather than globally coordinated second-order cone programming (SOCP) refinements.

The results of these experiments are shown in Figure 7. Surprisingly, the measured runtime grows well below $O(n \log n)$, indicating sublinear scaling with respect to the number of circles.

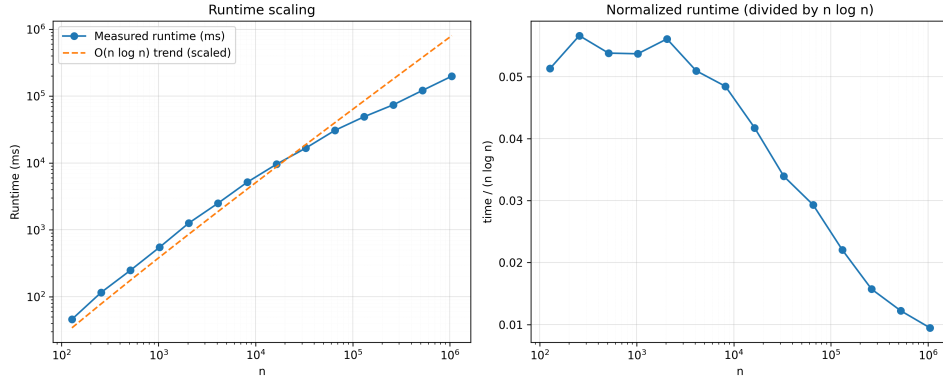


Figure 7: Runtime scaling on random CETSP instances. Runtime grows slower than $O(n \log n)$ due to sublinear growth of the tour size.

Investigation of the results revealed the reason: the number of tour points increases sublinearly with the number of input circles. For example, an instance with 2^{19} circles produced a tour of length 20,246, while 2^{20} circles resulted in a tour length of 24,307. Consequently, most of the $n \log n$ component of the solver runtime is effectively scaled down by the sublinear growth of the tour.

5.2 Structured Input Experiments

To remove this effect and demonstrate the asymptotic scaling clearly, we generated structured test instances that guarantee a linear tour size. For each n , we constructed a 2D grid of size $\lfloor \sqrt{n} \rfloor \times \lfloor \sqrt{n} \rfloor$, with centers placed at the grid points plus a small random jitter of ± 0.1 . Any additional points were placed uniformly at random within the grid area. The circle radii were

sampled uniformly from $[0.2, 0.5]$. For these instances, we performed a single repetition of the solver for efficiency.

The results, shown in Figure 8, exhibit a clear $O(n \log n)$ scaling, confirming empirically that the solver runtime behaves as expected when the tour size scales linearly with n .

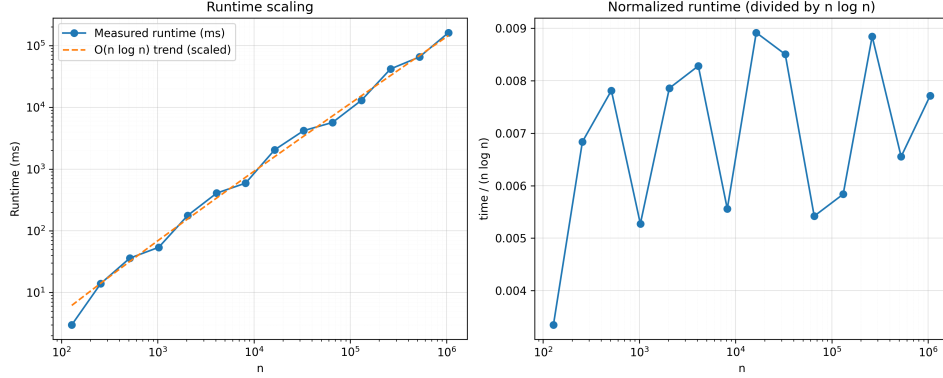


Figure 8: Runtime scaling on structured CETSP instances with linear tour size. The $O(n \log n)$ trend fits the measured data closely.

5.3 Conclusion

In summary, our experiments confirm that the solver exhibits $O(n \log n)$ runtime behavior in the regime where the tour size scales linearly with the number of input circles. On purely random instances, the observed runtime is often sub- $O(n \log n)$ due to sublinear growth of the tour size, but the structured experiments demonstrate that the underlying algorithmic complexity follows the expected scaling law.

A Appendix: Table of Results

Table 1: Comparison of benchmark vs output values for all instances

Instance	N	OR%	Benchmark Value	Output Value ²	Gap%
bonus1000 ³	1000	12.26	384.365	378.622	-1.49
bonus1000rdmRad	1000	6.13	917.610	937.247	2.14
bubbles1	36	11.11	349.135	349.255	0.03
bubbles2	76	9.09	428.279	428.368	0.02
bubbles3	126	7.69	529.955	531.243	0.24
bubbles4	184	6.67	802.974	809.216	0.78
bubbles5	250	5.88	1035.320	1055.488	1.95
bubbles6	324	5.26	1220.070	1301.459	6.67
bubbles7	406	4.76	1575.040	1647.837	4.62
bubbles8	496	4.35	1881.930	1995.234	6.02
bubbles9	594	4.00	2148.400	2331.431	8.52
chaoSingleDep	200	2.84	1039.610	1042.864	0.31
concentricCircles1	16	15.00	53.158	53.201	0.08
concentricCircles2	36	7.50	153.132	153.870	0.48
concentricCircles3	60	5.00	270.007	271.744	0.64
concentricCircles4	104	3.75	451.870	460.079	1.82
concentricCircles5	148	3.00	632.977	639.080	0.96
d493_or10	493	10.00	100.721	100.959	0.24
d493_or2	493	2.00	199.015	200.798	0.90
d493_or30	493	30.00	69.758	69.758	0.00
d493rdmRad	493	13.44	134.226	134.621	0.29
dsj1000_or10	1000	10.00	373.759	376.445	0.72
dsj1000_or2	1000	2.00	909.502	922.440	1.42
dsj1000_or30	1000	30.00	199.948	200.037	0.04
dsj1000rdmRad	1000	12.47	624.604	626.148	0.25
kroD100_or10	100	10.00	89.668	89.781	0.13
kroD100_or2	100	2.00	159.037	159.603	0.36
kroD100_or30	100	30.00	58.541	58.576	0.06
kroD100rdmRad	100	4.03	141.829	142.199	0.26
lin318_or10	318	10.00	1394.630	1404.343	0.70
lin318_or2	318	2.00	2816.580	2833.410	0.60
lin318_or30	318	30.00	765.964	765.998	0.00
lin318rdmRad	318	10.05	2047.110	2052.486	0.26
pcb442_or10	442	10.00	142.573	143.302	0.51
pcb442_or2	442	2.00	319.846	332.701	4.02
pcb442_or30	442	30.00	83.537	83.538	0.00
pcb442rdmRad	442	9.39	219.219	220.280	0.48
rat195_or10	195	10.00	67.991	68.080	0.13
rat195_or2	195	2.00	157.970	160.495	1.60
rat195_or30	195	30.00	45.702	45.702	0.00
rat195rdmRad	195	42.60	68.224	68.252	0.04
rd400_or10	400	10.00	452.826	453.943	0.25
rd400_or2	400	2.00	1018.460	1040.517	2.17
rd400_or30	400	30.00	224.839	224.910	0.03
rd400rdmRad	400	0.99	1238.280	1279.515	3.33

Continued on next page

Table 1 – continued from previous page

Instance	N	OR%	Benchmark Value	Output Value	Gap%
rotatingDiamonds1	20	20.00	32.389	32.431	0.13
rotatingDiamonds2	60	5.00	140.477	140.636	0.11
rotatingDiamonds3	180	3.33	380.882	381.475	0.16
rotatingDiamonds4	320	1.43	770.661	790.601	2.59
rotatingDiamonds5	680	1.11	1510.750	1554.064	2.87
team1_100	100	9.33	307.337	308.339	0.33
team1_100rdmRad	100	7.69	388.537	399.272	2.76
team2_200	200	20.06	246.683	247.401	0.29
team2_200rdmRad	200	5.19	613.659	618.001	0.71
team3_300	300	7.02	461.889	462.960	0.23
team3_300rdmRad	300	23.70	378.087	379.137	0.28
team4_400	400	5.01	669.910	674.344	0.66
team4_400rdmRad	400	2.05	984.240	1013.142	2.94
team5_499	499	2.00	693.798	704.900	1.60
team5_499rdmRad	499	20.14	446.191	446.525	0.07
team6_500	500	27.06	225.216	225.451	0.10
team6_500rdmRad	500	10.02	620.886	623.678	0.45

Table 2: Runtime comparison: benchmark vs our run (adjusted)

Instance	Benchmark Time (s)	Our time (s)	Adjusted time (s) ⁴	Adj / Bench ⁵
bonus1000	2016.81	0.300	3.127	0.0016
bonus1000rdmRad	890.66	0.239	2.491	0.0028
bubbles1	31.12	0.014	0.146	0.0047
bubbles2	41.73	0.025	0.261	0.0062
bubbles3	193.23	0.051	0.532	0.0028
bubbles4	173.63	0.077	0.803	0.0046
bubbles5	247.52	0.125	1.303	0.0053
bubbles6	358.74	0.192	2.001	0.0056
bubbles7	650.29	0.228	2.377	0.0037
bubbles8	790.10	0.292	3.044	0.0039
bubbles9	893.73	0.443	4.618	0.0052
chaoSingleDep	76.44	0.059	0.615	0.0080
concentricCircles1	29.31	0.002	0.021	0.0007
concentricCircles2	139.32	0.011	0.115	0.0008
concentricCircles3	67.01	0.023	0.240	0.0036
concentricCircles4	193.29	0.049	0.511	0.0026
concentricCircles5	208.92	0.082	0.855	0.0041
d493_or10	352.52	0.033	0.344	0.0010
d493_or2	689.44	0.140	1.459	0.0021
d493_or30	116.74	0.267	2.783	0.0238
d493rdmRad	71.85	0.189	1.970	0.0274
dsj1000_or10	942.49	0.089	0.928	0.0010
dsj1000_or2	2578.76	0.240	2.502	0.0010
dsj1000_or30	249.61	0.597	6.223	0.0249
dsj1000rdmRad	176.72	0.239	2.491	0.0141

Continued on next page

²Output values obtained using our CETSP solver over 1000 runs, the best value is reported.³The **bonus1000** test case was ran 10000 times to improve the best value.

Table 2 – continued from previous page

Instance	Benchmark Time (s)	Our time (s)	Adjusted time (s)	Adj / Bench
kroD100_or10	50.76	0.030	0.313	0.0062
kroD100_or2	158.32	0.020	0.208	0.0013
kroD100_or30	39.25	0.038	0.396	0.0101
kroD100rdmRad	158.19	0.009	0.094	0.0006
lin318_or10	193.10	0.047	0.490	0.0025
lin318_or2	327.54	0.094	0.981	0.0030
lin318_or30	71.47	0.199	2.076	0.0290
lin318rdmRad	66.44	0.059	0.615	0.0093
pcb442_or10	467.38	0.082	0.855	0.0018
pcb442_or2	1334.95	0.033	0.344	0.0003
pcb442_or30	117.86	0.140	1.459	0.0124
pcb442rdmRad	160.56	0.267	2.783	0.0173
rat195_or10	170.90	0.189	1.970	0.0115
rat195_or2	322.37	0.089	0.928	0.0029
rat195_or30	50.19	0.240	2.502	0.0499
rat195rdmRad	32.17	0.597	6.223	0.1934
rd400_or10	540.53	0.239	2.491	0.0046
rd400_or2	450.63	0.030	0.313	0.0007
rd400_or30	91.67	0.020	0.208	0.0023
rd400rdmRad	635.69	0.038	0.396	0.0006
rotatingDiamonds1	33.70	0.059	0.615	0.0182
rotatingDiamonds2	46.81	0.082	0.855	0.0183
rotatingDiamonds3	87.69	0.105	1.094	0.0125
rotatingDiamonds4	229.92	0.302	3.149	0.0137
rotatingDiamonds5	597.20	0.073	0.761	0.0013
team1_100	54.29	0.009	0.094	0.0017
team1_100rdmRad	38.02	0.050	0.521	0.0137
team2_200	181.62	0.122	1.270	0.0070
team2_200rdmRad	93.86	0.022	0.230	0.0024
team3_300	435.69	0.287	2.992	0.0069
team3_300rdmRad	48.46	0.133	1.389	0.0287
team4_400	648.08	0.273	2.844	0.0044
team4_400rdmRad	734.98	0.055	0.574	0.0008
team5_499	561.82	0.003	0.031	0.0001
team5_499rdmRad	51.36	0.026	0.271	0.0053
team6_500	208.22	0.114	1.188	0.0057
team6_500rdmRad	261.21	0.229	2.384	0.0091

⁴Our runtimes are measured on an AMD Ryzen 9 7940HS and reported in seconds. Times are scaled to the benchmark CPU (AMD Opteron 4184) using a PassMark-based factor of 10.424 (29938 / 2872), following **LeiHao2024**.

⁵Adj / Bench = Adjusted time / Benchmark time.